



WHITEPAPER

Project Ironroot

Mitigating OS-Level Security Risks from SAP Application
Access with **SELinux** and **fapolicyd**

October 5th 2026

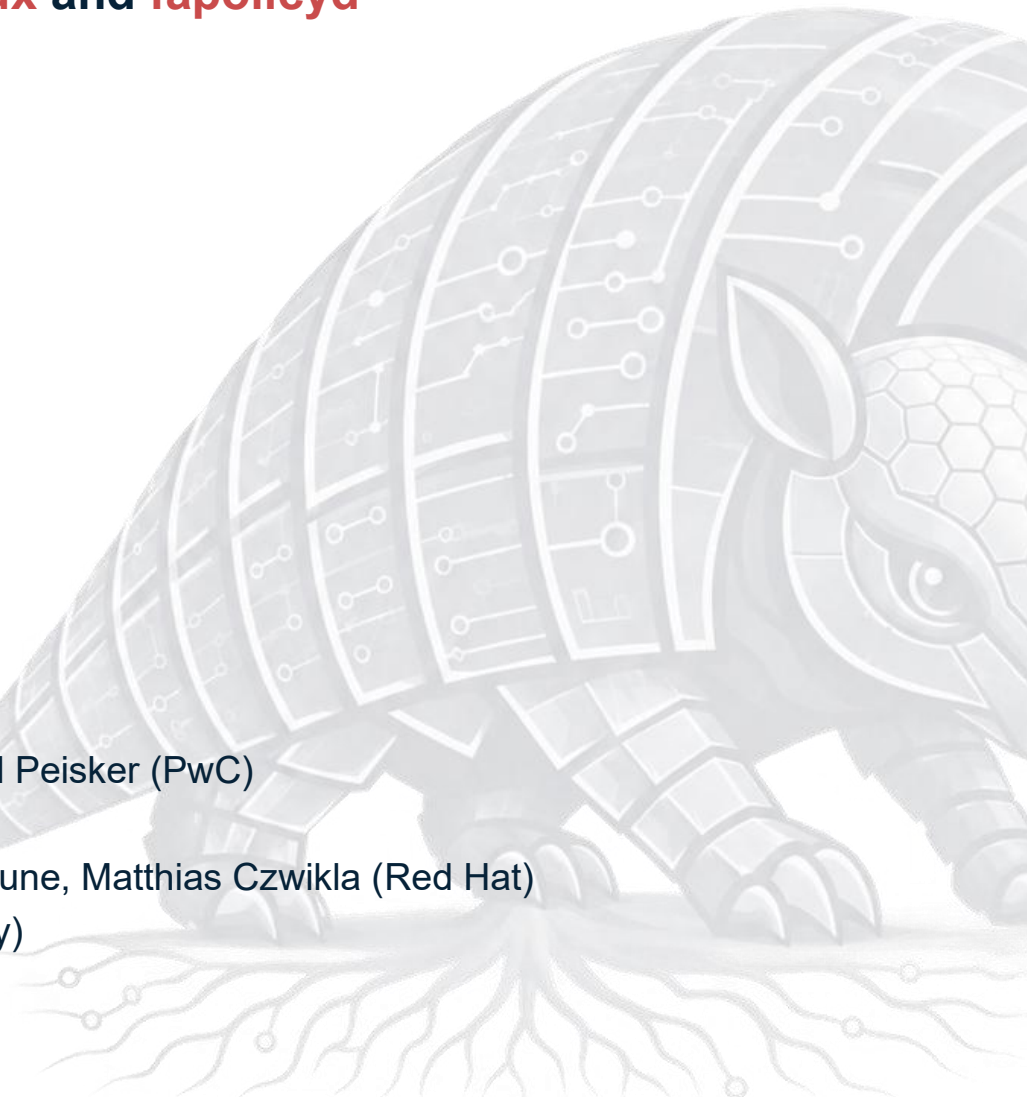
Contributors

Benjamin Fani Sani, Daniel Peisker (PwC)

Julian Peterson (Fortinet)

Bernd Finger, Christoph Brune, Matthias Czwikla (Red Hat)

Waseem Arjab (No Monkey)



Disclaimer

This whitepaper provides technical guidance based on collaborative industry research and findings from the “Project Ironroot” workshop. It is intended to complement, not replace, official vendor documentation, including SAP Notes, the SAP Security Baseline, and Linux vendor hardening guides.

The security controls and tools described in this whitepaper can have a significant impact on host systems and SAP application operations, including system availability, performance, functionality, and vendor supportability. All configurations must therefore be carefully evaluated, adapted to the specific environment, and thoroughly tested in non-production systems before implementation.

Feature availability and implementation details for tools such as SELinux and fapolicyd vary across Linux distributions. Security and operations teams must select and configure controls according to the requirements and capabilities of their chosen operating system and SAP environment.

The contributors assume no liability for any system impact, including outages, performance degradation, loss of functionality, or security incidents, arising from the use of the described tools or the implementation of these recommendations.

List of Contents

1. Introduction	1
2. Technical Context	2
2.1. SAP System Access to Operating System Commands and File Operations	2
2.2. How SAP Executes Code on a Server	2
2.3. Technical Mechanisms for Executing Operating System Commands	3
2.4. Conceptual Execution Flow from ABAP to the Operating System	4
2.5. File System Access as a First-Class SAP Capability	5
2.6. A Word about RISE with SAP deployments	5
2.7. Summary	6
3. Security Risk Posture	7
3.1. What Could Go Wrong	7
3.2. Attack Path Expansion Beyond SAP	8
3.3. Security Implications	8
3.4. Practical Attack Scenarios	9
3.4.1. Arbitrary File Read	9
3.4.2. Arbitrary File Write	11
3.4.3. Executing Binaries	12
3.4.4. Establishing an Operating System Reverse Shell	15
4. Defense in Depth and Blast Radius Reduction	17
4.1. Why Prevention alone is Insufficient	17
4.2. The Philosophy of the Following Guidance	17
4.3. Security Objectives	18
5. Security Risk Mitigation Strategies	19
5.1. File System Permissions and Ownership	19
5.2. SELinux	19
5.3. fapolicyd	20
5.4. How the Controls Work Together	21
5.5 From Guidance to Pilot Implementation	21
6. Technical Implementation of SELinux and fapolicyd	22
6.1. SAP-Specific SELinux Policy	23
6.2. Extending Enforcement to SAP Administrator Sessions	23
6.3. Mitigating Arbitrary File Read	24
6.4. Mitigating Arbitrary File Write	25
6.5. Restricting Execution	29
6.6. Reducing the Executable Attack Surface	30
6.7. Limiting Reverse-Shell Capabilities	31
6.8. Operational Considerations	31
6.9. Resulting Security Architecture	32
7. Conclusion	33

List of Figures

Figure 1: SAP-to-operating-system execution flow and trust boundary through SAP Kernel.....	4
Figure 2: Creation of a custom AL11 directory parameter pointing to /etc.....	9
Figure 3: Browsing the contents of the Linux /etc directory through AL11.....	10
Figure 4: Displaying the contents of /etc/passwd directly within the SAP GUI.....	10
Figure 5: Successful execution of a custom ABAP report that writes a file to the operating system.....	12
Figure 6: Definition of an external operating system command in SM69.....	13
Figure 7: Successful execution of the configured command through SM49.....	14
Figure 8: Command execution through RSBDCOS0, including validation of the execution context and reverse shell initiation.....	15
Figure 9: Reverse shell connection received on the attacker-controlled system, providing interactive access to the SAP host.	15
Figure 10: SELinux blocks an ABAP OPEN DATASET read of a HANA resource	25
Figure 11: ABAP OPEN DATASET logic used to test file read access	27
Figure 12: ABAP test program configured for a read operation against a temporary file	27
Figure 13: Successful ABAP read of a temporary file created by the HANA administrator.....	28
Figure 14: ABAP OPEN DATASET logic used to test file write access.....	28
Figure 15: ABAP test program configured for a write operation against a temporary file	28
Figure 16: ABAP write operation blocked with Permission denied under the SELinux policy	29

List of Tables

Table 1: Comparison of Linux Security Controls for SAP Blast-Radius Reduction.....	21
--	----

List of Listings

Listing 1: ABAP report for writing a file to the operating system.....	12
Listing 2: Example SELinux AVC denial event.....	20
Listing 3: strace command proposed for SAP file-access baseline analysis	24
Listing 4: Pre-policy HANA file access from NetWeaver administrator accounts.....	24
Listing 5: Post-policy denial of NetWeaver access to HANA log resources.....	25
Listing 6: SELinux temporary-file labeling and cross-domain write-denial test	26
Listing 7: fapolicyd enablement and trust-database validation commands.....	30

Abbreviations

Abbreviation / Identifier	Meaning
ABAP	Advanced Business Application Programming - SAP programming language
ACL	Access Control List
AL11	SAP transaction for displaying operating-system directories
ASCS	ABAP SAP Central Services
AVC	Access Vector Cache - SELinux access-decision logging
DAC	Discretionary Access Control
ERS	Enqueue Replication Server
GUI	Graphical User Interface
HANA	SAP HANA in-memory database platform
MAC	Mandatory Access Control
OS	Operating System
PID	Process Identifier
RFC	Remote Function Call
RHEL	Red Hat Enterprise Linux
RISE	RISE with SAP - managed cloud transformation offering
RSBDCOS0	SAP report for executing operating-system commands (deprecated)
SAP	Systems, Applications and Products in Data Processing
SE38	SAP ABAP Editor transaction
SELinux	Security-Enhanced Linux
SID	SAP System Identifier
SM49	SAP transaction for executing external operating-system commands
SM69	SAP transaction for maintaining external operating-system commands
SSH	Secure Shell
S/4HANA	SAP ERP suite running on SAP HANA
SXPG_COMMAND_EXECUTE	SAP function module for executing predefined external commands
TH_INFO	SAP kernel interface exposing system/process information
TLS	Transport Layer Security

1. Introduction

SAP systems occupy a unique position within enterprise environments. They process some of an organization's most critical business data and support core functions ranging from finance and procurement to manufacturing and logistics. To perform these tasks, SAP applications are interacting closely with the underlying operating system, including the ability to access files and execute operating system functions.

This tight integration is a fundamental characteristic of SAP architecture, but it also creates a significant trust relationship between the SAP application layer and the host operating system. If an attacker succeeds in exploiting a vulnerability, misconfiguration, or compromised account within the SAP stack he can gain access to the operating system, and while doing so bypass the security controls that normally protect SAP business processes.

This whitepaper deliberately assumes a compromise leading to privileged authorizations on application level has already occurred. Rather than focusing on how to prevent every possible attack, it examines how organizations can limit the consequences of a successful compromise. The objective is to reduce the attacker's freedom of action, contain potential damage, and prevent an isolated incident from escalating into a broader infrastructure-wide compromise.

To achieve this, we explore the use of operating-system-level security controls SELinux and fapolicyd. These technologies complement existing SAP security measures by introducing additional enforcement boundaries at the Linux layer, helping organizations reduce attack surface, contain compromised processes, and strengthen the overall resilience of their SAP landscape. This paper investigates in what ways SELinux and fapolicyd can reduce the impact of OS interaction via the SAP environment and where its limitations are.

The central message of this paper is straightforward: while a compromise of the SAP application cannot always be prevented, its impact on the underlying host systems and the broader IT infrastructure can be significantly reduced by applying key defense-in-depth principles.

2. Technical Context

2.1. SAP System Access to Operating System Commands and File Operations

To understand the security risks discussed in this paper, it is first necessary to understand how SAP systems interact with the underlying operating system. Many business-critical SAP functions depend on the ability to access files, execute operating system commands, and interact directly with host resources.

The following sections explain how these interactions are implemented and where the trust boundary between the SAP application layer and the operating system resides. This technical context provides the foundation for understanding the attack scenarios and mitigation strategies presented later in this paper.

Enterprise SAP systems are not isolated application stacks operating independently of their host environment. Instead, they are designed as long-running, stateful platforms that execute continuously on dedicated servers and rely on deep integration with the underlying operating system. This architectural model predates modern containerization and reflects decades of enterprise operational requirements.

In practice, an SAP system must be able to interact with the operating system in order to function correctly. Routine activities such as writing log files, processing interface data, executing background jobs, importing transports, or performing system health checks all require access to operating system resources. As a result, SAP systems are intentionally designed with the ability to execute operating system commands and perform file system operations.

This capability is neither accidental nor exceptional; it is a foundational design element. However, because these interactions occur with the privileges of a powerful operating system account, they form a critical boundary between the SAP application layer and the host system. Understanding how this boundary is implemented is therefore essential before discussing security risk, abuse scenarios, or mitigation strategies.

This chapter establishes that understanding by explaining how SAP systems execute operating system commands, which technical mechanisms are involved, and how responsibilities are divided between the SAP runtime and the host operating system.

2.2. How SAP Executes Code on a Server

From the perspective of the operating system, an SAP system is a collection of processes that run under a dedicated operating system user. On Linux-based systems, this user is typically named after the SAP system identifier (for example, <sid>adm). All SAP application server processes - including dialog work processes, background job processors, update tasks, and internal services - run under this single operating system identity.

The operating system has no native understanding of SAP-specific concepts such as SAP users, roles, transactions, or authorizations. To the OS, all activity originating from SAP appears as actions performed by the SAP system user. This distinction becomes central when examining how operating system commands are executed.

Between the ABAP application code and the operating system sits the SAP kernel. The kernel is implemented in platform-specific native code and is responsible for all low-level interactions with the host environment. It manages process creation, memory handling, network communication, file system access, and the execution of operating system commands. ABAP cannot perform OS system calls or spawn OS processes itself; it can only request them through kernel interfaces and therefore relies on the kernel to do so.

This separation is a deliberate architectural choice. It allows SAP applications to remain portable across operating systems while centralizing control over all OS-level interactions within the kernel.

2.3. Technical Mechanisms for Executing Operating System Commands

SAP systems provide several mechanisms through which operating system commands can be executed. While these mechanisms differ in abstraction and historical context, they all follow the same fundamental pattern: an ABAP program expresses intent, the SAP kernel validates the request, and the kernel executes the command on the operating system using the SAP system's OS user.

The most common and officially supported mechanism is based on predefined external commands. In this model, administrators explicitly register which operating system commands may be executed by the SAP system. ABAP programs invoke these commands using standardized function modules, such as `SXPG_COMMAND_EXECUTE`. When such a function is called, the SAP kernel resolves the configured command, evaluates authorizations, handles parameters, and performs the actual execution at the operating system level. This mechanism is widely used for administrative automation, monitoring, and integration tasks, and represents SAP's preferred approach for controlled OS interaction.

In addition to these high-level interfaces, SAP historically exposed lower-level kernel interfaces that allow ABAP programs to trigger operating system commands more directly. Examples include ABAP statements such as `CALL 'SYSTEM'` or `CALL 'CFUNC'`, which pass command strings directly to kernel routines. These statements originate from earlier SAP system generations and are primarily encountered in legacy or long-lived custom code. Although their use is strongly discouraged and increasingly restricted in modern systems, they remain part of the ABAP language and are relevant for understanding the full execution model.

Operating system commands may also be executed as part of internal system services rather than directly from application code. SAP kernel components regularly invoke OS-level functionality to perform tasks such as instance control, system status checks, or resource monitoring. Interfaces such as `TH_INFO` and related kernel calls expose system and process information to the SAP runtime by querying the operating system on behalf of the application. In

these cases, the execution is initiated internally by the SAP system itself, but the underlying execution context remains unchanged.

Across all these mechanisms, one constant remains: the operating system command is executed by the SAP kernel, under the SAP system's operating system user.

2.4. Conceptual Execution Flow from ABAP to the Operating System

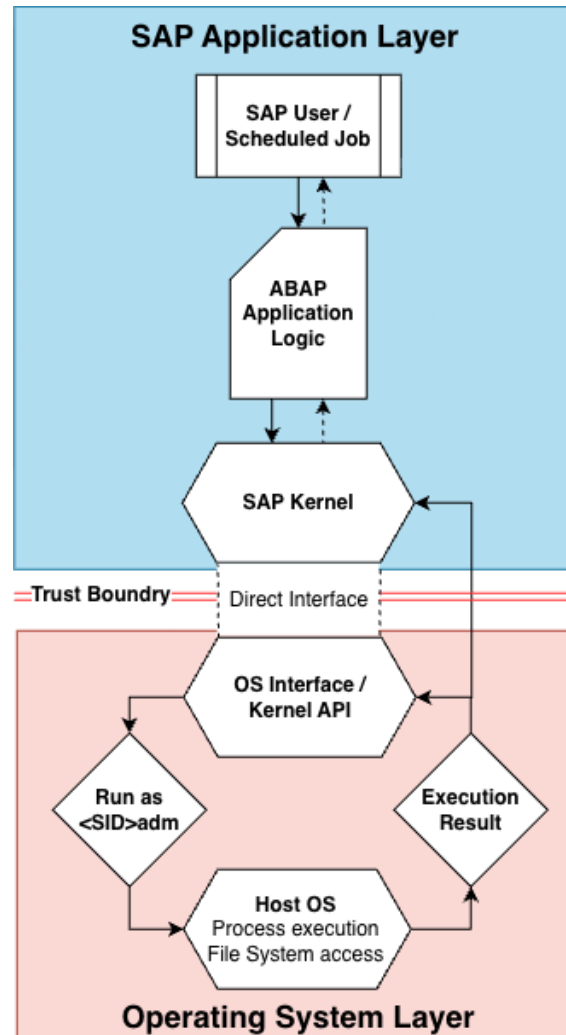


Figure 1: SAP-to-operating-system execution flow and trust boundary through SAP Kernel.

Conceptually, the execution of an operating system command from SAP follows a well-defined flow. An SAP user action, background job, or interface process triggers ABAP application logic. That logic issues a request to perform an operating system action, such as executing a command or accessing a file. The request is passed to the SAP kernel, which validates it, resolves the command or file path, and invokes the corresponding operating system interface. The operating system executes the request using the SAP system's OS user, and the result is returned through the kernel back to the ABAP runtime.

From the perspective of the operating system, all such activity is indistinguishable: it appears as process creation or file access performed by the SAP OS user, regardless of which SAP user or application context initiated the action.

A notable consequence of this architecture is a clear separation of visibility. At the SAP application level, operating system interactions are highly abstracted. Applications typically receive only a return code and, optionally, standard output or error messages. Details about which binaries were executed, which files were accessed, or which system calls were performed are not visible within ABAP.

At the operating system level, the situation is reversed. Administrators can observe processes, file accesses, and resource usage performed by the SAP OS user, but the operating system has no inherent knowledge of which SAP user, transaction, or business process initiated a given action. This disconnect between application intent and host-level execution is an inherent characteristic of SAP's execution model.

2.5. File System Access as a First-Class SAP Capability

Executing operating system commands is only one aspect of SAP's interaction with the host environment. Equally important is extensive file system access. SAP systems continuously read from and write to the file system to manage logs, traces, interface files, data exports, and lifecycle artifacts such as transports.

These file operations follow the same architectural principles as command execution. Requests originate in ABAP or kernel logic, but the SAP kernel performs the actual file system interaction. Access rights are governed exclusively by the permissions assigned to the SAP OS user and its associated groups. As a result, the effective file system authority of an SAP system is determined not by SAP-level authorizations, but by operating system permissions.

Intentional by Design

It is important to emphasize that SAP's ability to execute operating system commands and perform file operations is an intentional and necessary design feature. Many production systems rely on this functionality for stable operation, automation, and integration. Custom extensions and customer-specific tooling often depend on the same mechanisms.

The presence of this functionality alone does not imply insecurity. Its significance lies in how it is governed, constrained, and monitored - topics that are addressed in the subsequent chapters of this paper.

2.6. A Word about RISE with SAP deployments

The focus of this whitepaper is the technical relationship between SAP applications and the underlying operating system. In customer-operated environments, organizations typically have direct control over the operating system and can implement many of the controls discussed throughout this paper themselves.

In RISE environments, responsibility for operating system administration and infrastructure-level hardening largely resides with SAP and not with the customer. Customers generally do not have

direct access to the operating system layer and therefore cannot implement many of the technical measures described in this document themselves.

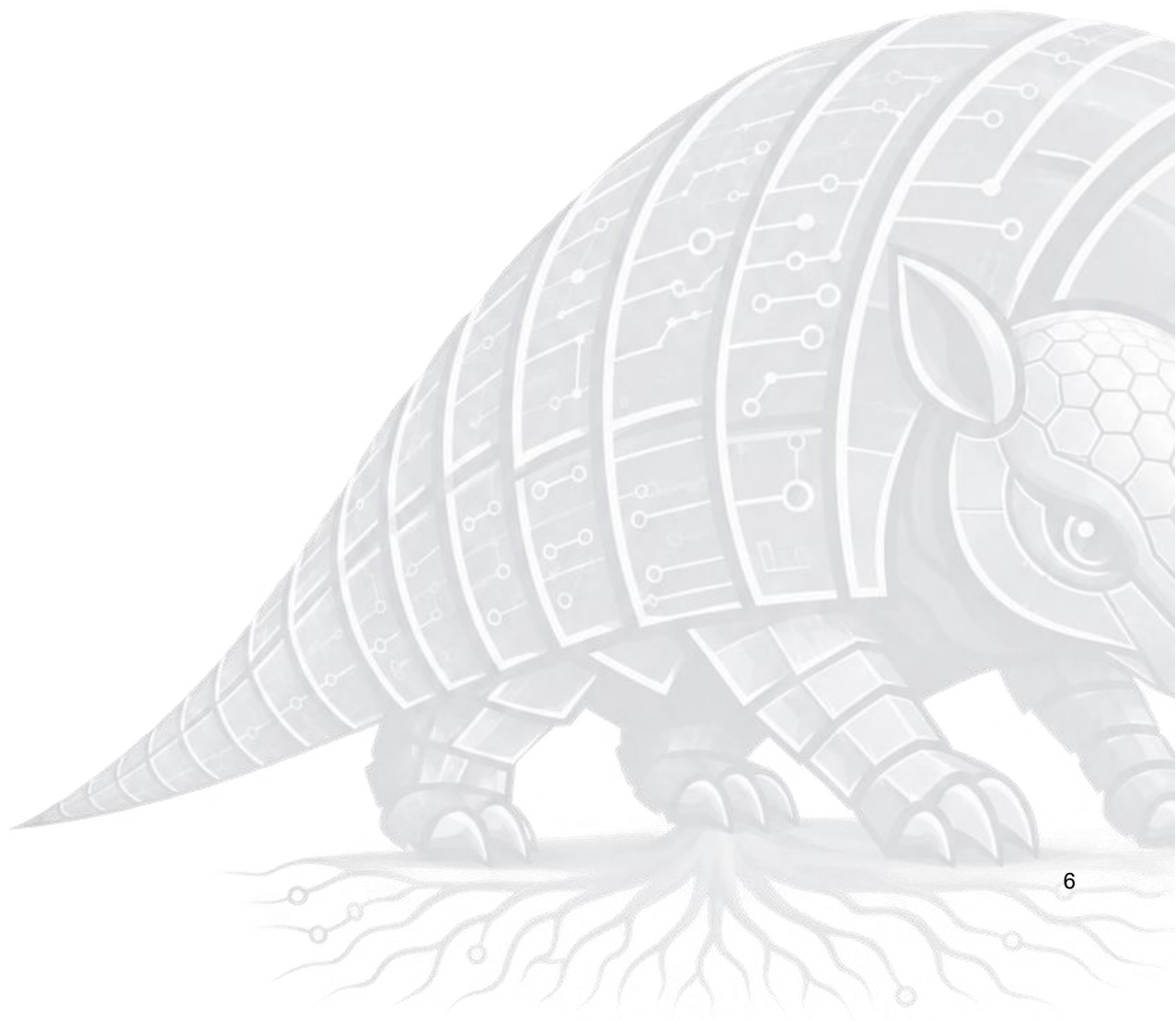
However, the architectural concepts discussed throughout this paper remain equally relevant. SAP applications continue to interact with the operating system through kernel-mediated interfaces, execute operating system commands, and perform file-system operations as part of normal system operation. The trust boundary between the SAP application layer and the host operating system therefore continues to exist, regardless of whether SAP operates the underlying infrastructure, a third party or if it is self-managed.

While the responsibility for implementing specific controls may differ between deployment models, the architectural principles, trust boundaries, and potential impact of operating system access remain relevant across both traditional and cloud-operated SAP landscapes.

2.7. Summary

SAP systems are architected to interact closely with their host operating systems through kernel-mediated execution paths. Operating system commands and file system operations are executed by the SAP kernel under a shared OS execution context, abstracted from application logic and SAP user identities.

By establishing a clear and accessible understanding of how these mechanisms work, this chapter provides the conceptual baseline required for the analysis of security risks, attack paths, and mitigation strategies that follow.



3. Security Risk Posture

The previous chapter explained how SAP systems interact with the underlying operating system and how operating system commands and file operations are executed through the SAP kernel. While these capabilities are essential for normal system operation, they also create a powerful trust relationship between SAP and the host operating system.

From a security perspective, the significance of this trust relationship becomes apparent once an attacker gains the ability to execute operating system commands or access files through the SAP stack. At that point, the attacker is no longer limited to SAP transactions, authorization objects, or application-layer functionality. Instead, they can interact directly with the operating system using the privileges available to the SAP operating system account.

The following sections examine the potential consequences of such access and how a compromise of a single SAP system can affect the wider IT landscape.

3.1. What Could Go Wrong

The impact of operating system access depends heavily on the specific SAP landscape, operating system configuration, and surrounding infrastructure. However, several common attack scenarios can be observed across many environments.

One of the most immediate consequences is access to sensitive files that are required for SAP operation. Depending on the deployed components, these may include configuration files, certificates, cryptographic material, RFC destinations, database connection information, SAProuter configurations, interface definitions, and other files that contain information useful for further compromise.

Access to the file system may also allow attackers to manipulate SAP-related content. Examples include modifying start scripts, changing configuration parameters, altering transport files, replacing custom scripts, or tampering with files that are processed by interfaces and background jobs. Since these changes occur at the operating system level, they may not be visible through traditional SAP authorization concepts.

Another important consideration is persistence. Once operating system access has been obtained, attackers may attempt to establish mechanisms that survive password changes or application-level remediation efforts. This can include modifications to startup scripts, scheduled tasks, service definitions, or other operating system components that are executed automatically.

Operating system access also provides opportunities for defense evasion. Log files may be altered or deleted, forensic evidence may be removed, and monitoring mechanisms may be disrupted. While the exact possibilities depend on the permissions available to the SAP operating system account, the ability to interact directly with the host significantly expands the attack surface compared to a compromise that remains confined to the application layer.

Finally, operating system access often enables the execution of additional tooling. Attackers frequently upload scripts, administrative utilities, reconnaissance tools, tunneling software, or

custom malware to facilitate further operations. In many environments, standard Linux permissions alone provide little resistance against such activities once a trusted account has been compromised.

3.2. Attack Path Expansion Beyond SAP

The impact of operating system access is rarely limited to the compromised host itself.

SAP systems are typically integrated with a large number of external services and infrastructure components. Databases, backup solutions, directory services, monitoring platforms, virtualization environments, file shares, and management systems frequently maintain trusted relationships with SAP servers. In many cases, credentials required for these integrations are stored on the operating system or are accessible to SAP-related processes.

As a result, a compromise of the SAP operating system account can become a starting point for further attacks against connected systems. Attackers may leverage stored credentials, abuse trusted communication paths, or access shared resources that were originally intended for operational purposes.

Particularly common examples include movement between SAP systems through shared transport directories, access to shared file systems, compromise of backup infrastructure, or the abuse of technical accounts used for integrations and automation. In environments where multiple SAP systems share administrative concepts or operating system groups such as sapsys, the compromise of a single host may expose resources belonging to multiple systems.

The security risk therefore extends beyond the confidentiality and integrity of the affected SAP instance. A successful compromise may provide opportunities to access additional systems, credentials, and trust relationships throughout the wider enterprise environment.

3.3. Security Implications

A recurring observation in many SAP environments is that the SAP operating system account possesses significantly broader access than is strictly required for individual business processes. This is often the result of operational requirements, historical configurations, shared administration concepts, and the need to maintain compatibility across complex SAP landscapes.

Traditional Linux file permissions provide only limited protection in such scenarios. Once an attacker operates within the security context of a trusted SAP account, the operating system generally treats all actions as legitimate activity performed by that account. The distinction between normal SAP operation and malicious activity becomes increasingly difficult to enforce through discretionary access controls alone.

This creates a gap between application-layer security and operating-system-layer security. While SAP authorizations can restrict what users are allowed to do inside the application, they cannot directly restrict what a compromised operating system account can do once access to the host has been obtained.

Closing this gap is the primary objective of the mitigation strategies discussed in the following chapters.

3.4. Practical Attack Scenarios

The security risks discussed in the previous chapters are not purely theoretical. To better illustrate the practical implications of operating system access obtained through the SAP stack, a series of representative attack scenarios were performed in a controlled test environment.

The objective of these examples is not to demonstrate specific vulnerabilities, but to show how an attacker can leverage the privileges of the SAP operating system account after a compromise has occurred. The scenarios highlight common post-compromise activities and demonstrate how seemingly limited access can be used to access sensitive information, establish persistence, move laterally, or prepare further attacks against connected systems.

The examples presented in this section are based on realistic SAP operating environments and are intended to illustrate the types of activities that organizations should consider when assessing the potential impact of a compromise.

3.4.1. Arbitrary File Read

The transaction AL11 is intended to provide administrators with access to predefined operating system directories required for SAP operation. The accessible directories are defined through SAP directory parameters and can be browsed directly from within the SAP GUI.

If an attacker is able to create or modify directory definitions, AL11 can be abused to access arbitrary locations on the underlying operating system. In the demonstrated scenario, a new directory parameter was created that pointed to the Linux directory /etc (Figure 2). After opening the directory in AL11, the contents of the operating system directory became fully visible through the SAP interface, including sensitive system files such as /etc/passwd (Figure 3).

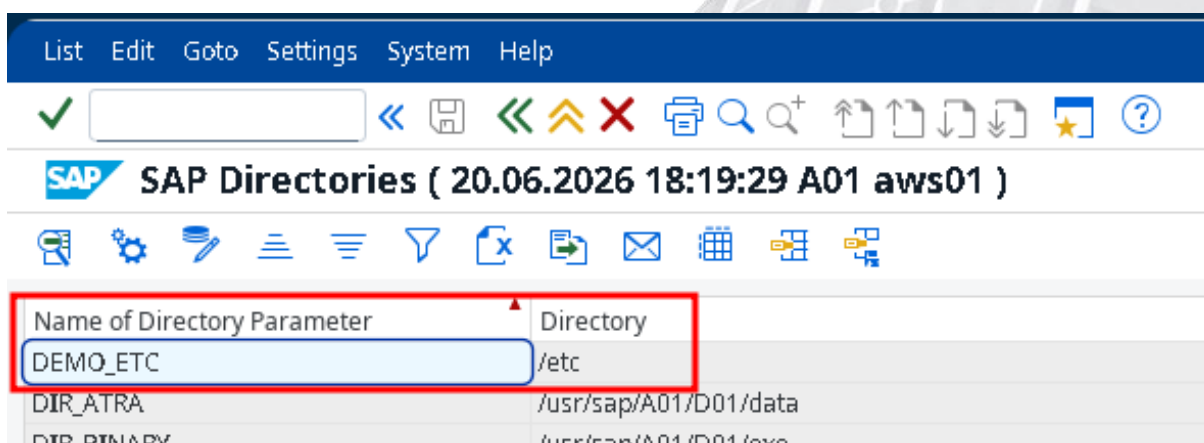
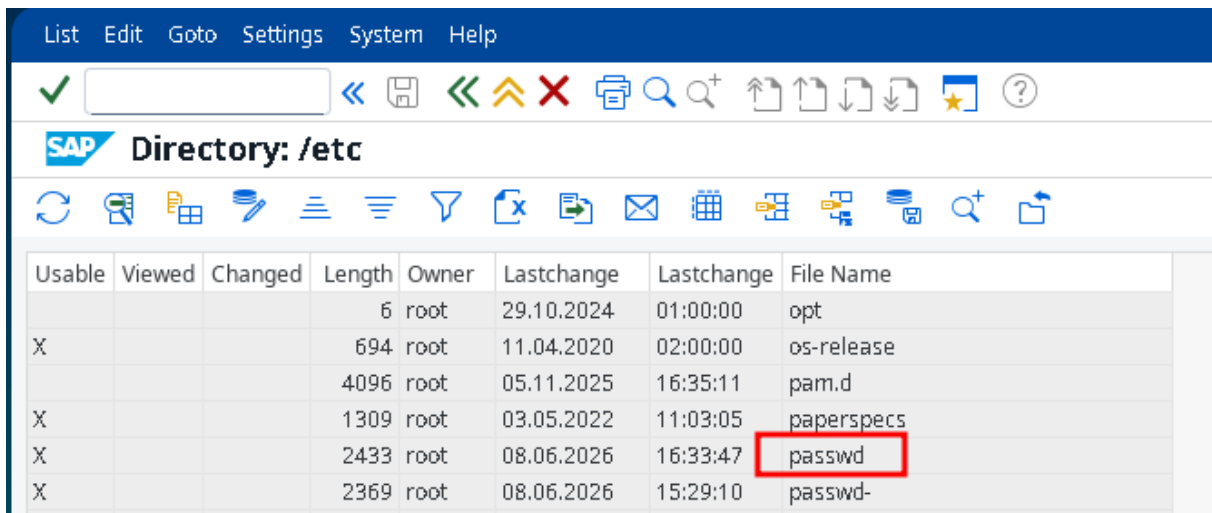


Figure 2: Creation of a custom AL11 directory parameter pointing to /etc.

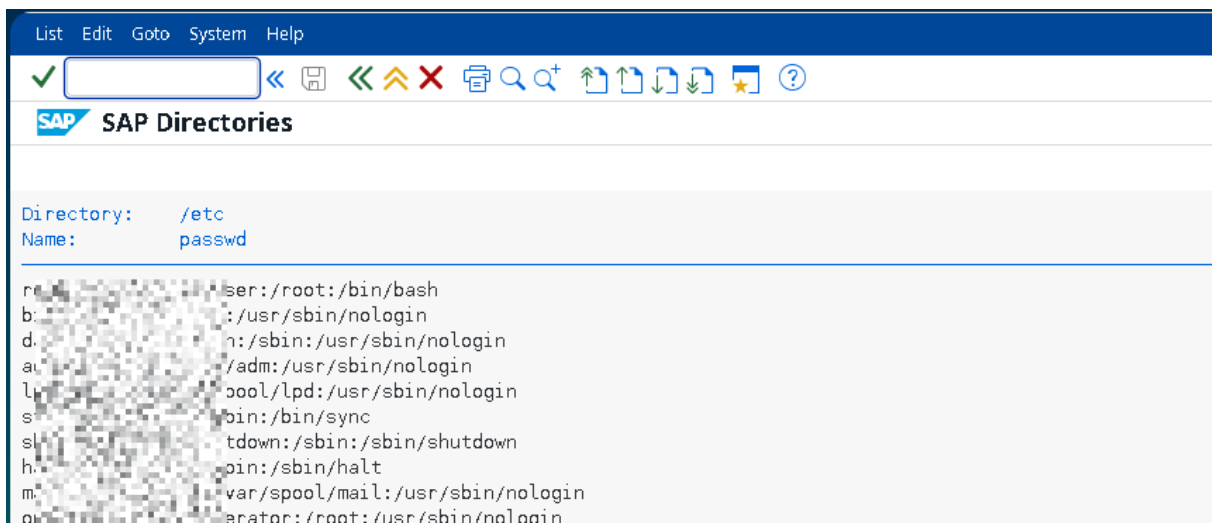
By selecting a file from the directory listing, SAP displays the file contents directly within the SAP GUI. As shown in Figure 4, the contents of /etc/passwd could be read without requiring direct shell access to the server.



Usable	Viewed	Changed	Length	Owner	Lastchange	Lastchange	File Name
			6	root	29.10.2024	01:00:00	opt
X			694	root	11.04.2020	02:00:00	os-release
			4096	root	05.11.2025	16:35:11	pam.d
X			1309	root	03.05.2022	11:03:05	paperspecs
X			2433	root	08.06.2026	16:33:47	passwd
X			2369	root	08.06.2026	15:29:10	passwd-

Figure 3: Browsing the contents of the Linux /etc directory through AL11.

This attack demonstrates that SAP authorization misuse can provide access to operating system files through standard SAP functionality. Depending on the permissions of the SAP application user on the host operating system, an attacker may be able to access configuration files, application data, log files, credentials, cryptographic material, or other sensitive information. While the example uses the publicly readable /etc/passwd file for demonstration purposes, the same technique can be used to access any file that is readable by the SAP operating system account.



SAP Directories	
Directory:	/etc
Name:	passwd
root:	/root:/bin/bash
bin:	/usr/sbin/nologin
d:	/sbin:/usr/sbin/nologin
adm:	/adm:/usr/sbin/nologin
lpd:	/pool/lpd:/usr/sbin/nologin
bin:	/bin/sync
shutdown:	/sbin:/sbin/shutdown
halt:	/sbin:/sbin/halt
mail:	/var/spool/mail:/usr/sbin/nologin
operator:	/root:/usr/sbin/nologin

Figure 4: Displaying the contents of /etc/passwd directly within the SAP GUI.

Although the example demonstrates access to /etc/passwd, the significance of this attack path extends far beyond the disclosure of a single operating system file. AL11 effectively provides a bridge between the SAP application layer and the underlying operating system. If an attacker can access arbitrary directories, they can enumerate and retrieve files that contain information valuable for further compromise.

On SAP systems, configuration files frequently contain details about system architecture, connected services, and security-relevant settings. SAP profile files may reveal instance configurations, trusted relationships, network endpoints, and operating system paths. Application

logs can expose usernames, hostnames, internal network information, and operational details that facilitate reconnaissance.

Depending on the operating system permissions of the SAP service account, additional sensitive files may also be accessible. These can include SSH keys, TLS certificates, database connection configurations, Secure Store files, application secrets, and other credentials used by SAP components. Such information can enable attackers to move beyond the SAP application itself and gain access to connected systems or privileged operating system accounts.

From a defensive perspective, this attack path highlights the importance of restricting access to transactions such as AL11, carefully controlling directory definitions, and limiting the file system permissions granted to SAP service accounts. Even when direct command execution is not possible, unrestricted file access can provide the information required to facilitate subsequent attack stages and significantly increase the impact of a compromise.

3.4.2. Arbitrary File Write

In addition to reading files from the operating system, SAP can also be used to create and modify files on the underlying host. ABAP provides native file handling functionality through the OPEN DATASET, TRANSFER, and CLOSE DATASET statements, allowing developers to interact directly with the file system from within SAP applications.

In the demonstrated scenario, a custom ABAP report was created that writes arbitrary content to a file on the operating system. The report was executed through transaction SE38 and successfully created the target file using the permissions of the SAP service account. Figure 5 shows the successful execution of the report within the SAP GUI.

The ability to write files to the operating system significantly increases the potential impact of an SAP compromise. While the example creates a harmless demonstration file, the same mechanism can be used to generate scripts, modify configuration files, plant malicious content, or prepare artifacts that may later be executed through other SAP functionality or administrative processes.

Unlike operating system access obtained through a shell, this attack path does not require direct access to the host itself. All actions are performed entirely through standard SAP functionality and are executed under the security context of the SAP operating system account.

```
1  REPORT Z_WRITE_OS_FILE_DEMO.
2  DATA lv_line TYPE string.
3  OPEN DATASET '/tmp/hello.sh'
4    FOR OUTPUT
5    IN TEXT MODE
6    ENCODING DEFAULT.
7  IF sy-subrc > 0.
8    WRITE: / 'Failed to open file'.
9    EXIT.
10 ENDIF.
11 lv_line = '#!/bin/sh'.
12 TRANSFER lv_line TO '/tmp/hello.sh'.
13 lv_line = 'echo "Hello from the OS"'.
14 TRANSFER lv_line TO '/tmp/hello.sh'.
15 CLOSE DATASET '/tmp/hello.sh'.
16 WRITE: / 'File written.'.
17
```

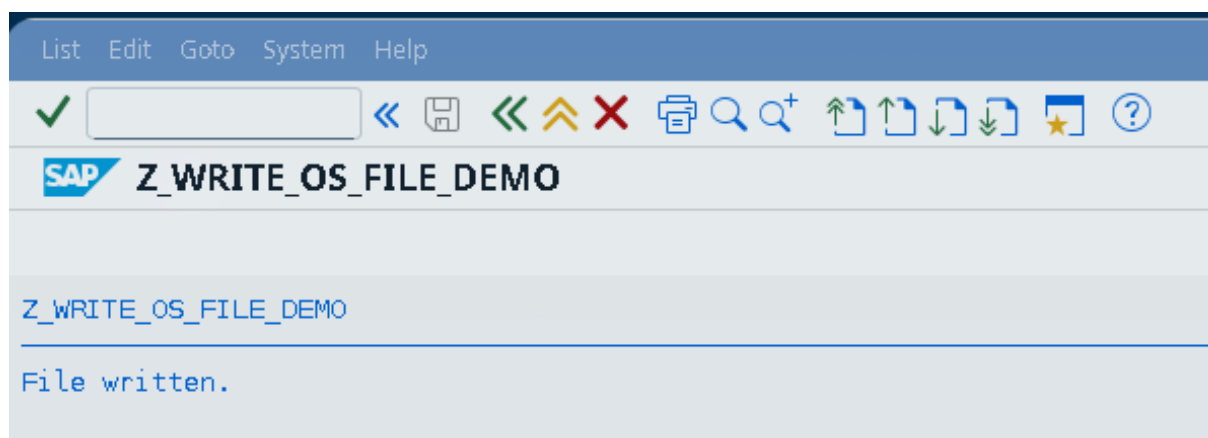


Figure 5: Successful execution of a custom ABAP report that writes a file to the operating system.

The ability to create arbitrary files on an SAP host provides attackers with a powerful primitive that can be combined with other attack techniques. Even in environments where direct command execution is restricted, file write access enables attackers to prepare payloads, manipulate application behavior, or establish persistence mechanisms.

For example, an attacker may create shell scripts, batch files, configuration files, scheduled task definitions, or other artifacts that are later processed by legitimate operating system services. In SAP environments, file write capabilities can also be used to influence interfaces, import processes, transport-related functionality, or custom integrations that consume files from specific directories.

The impact depends heavily on the permissions assigned to the SAP service account. If the account has access to security-relevant directories, attackers may be able to overwrite existing files, modify application behavior, or prepare a path toward operating system compromise. Even when write access is restricted to SAP-specific directories, the ability to create arbitrary files can facilitate data manipulation, persistence, or the staging of additional attack payloads.

3.4.3. Executing Binaries

SAP provides standard functionality for defining and executing external operating system commands. Transaction SM69 is used to create and maintain command definitions, while transaction SM49 can be used to execute those commands from within the SAP GUI.

In this scenario, an external command was created in SM69. The command definition points to /bin/sh and passes /tmp/hello.sh as a parameter, to execute the script that was created on the filesystem in the previous chapter. This configuration allows SAP to invoke the shell and execute the script located on the underlying operating system. Figure 6 shows the command definition created in SM69.

The command was then executed through SM49. SAP started the configured operating system command and returned the output directly in the SAP GUI. In this case, the script printed the

message Hello from the OS, confirming that code execution on the operating system was achieved through standard SAP functionality. Figure 7 shows the successful execution result.

Command

Command Name	Z_DEMO_SCRIPT
Operating System	Linux
Type	

Create and Last Change

Created By	
	00:00:00
Last Changed By	
	00:00:00

Definition

Operating System Command	/bin/sh
Parameters for Operating System Command	/tmp/hello.sh
☒ Additional parameters allowed	
☐ Trace	
Check Module	

Figure 6: Definition of an external operating system command in SM69.


Execute Command "Z_DEMO_SCRIPT" for "Linux"

Command

Command Name	Z_DEMO_SCRIPT	SAPXPG PID	551.502
Operating System	Linux	Conversation ID	64547533
Start Status	0	Stdin	R
Return Code	0	Stdout	M
Exit Code	0	Stderr	M
Exit Status	0	Wait for End	C
Execution Target	Trace Level 0		

Definition

Operating System Command

```

/bin/sh
/tmp/hello.sh

```











Hello from the OS

OVR Li 1, Co 18

Figure 7: Successful execution of the configured command through SM49.

External command execution is one of the most critical SAP-to-operating-system attack paths. It allows attackers to execute binaries, scripts, and system utilities with the privileges of the SAP service account. Depending on the permissions of that account, this can lead to system reconnaissance, data exfiltration, lateral movement preparation, persistence, or full compromise of the SAP host.

The security risk increases even more when command execution is combined with arbitrary file write access. An attacker can first write a script to the file system and then execute it through SM49. This creates a complete attack chain entirely within SAP: file creation, payload staging, and operating system execution.

3.4.4. Establishing an Operating System Reverse Shell

While SM69 and SM49 provide a structured mechanism for executing predefined operating system commands, SAP also contains functionality that allows direct interaction with the operating system shell. One example is the report RSBDCOS0, which executes operating system commands and displays the resulting output within the SAP GUI.

In the demonstrated scenario, RSBDCOS0 was used to execute commands directly on the SAP host. Initially, simple commands such as `whoami` were executed to verify operating system access and identify the context under which commands were running. The output confirmed that commands were executed as the SAP operating system user. Figure 8 shows the command history and execution output within RSBDCOS0.

After confirming command execution capabilities, a reverse shell was initiated using Netcat. The command instructed the SAP host to establish an outbound connection to an attacker-controlled system and redirect a shell through that connection. Once the connection was established, interactive command execution became possible from the attacker's system. Figure 9 shows the listener receiving the connection and the subsequent command execution through the interactive shell.

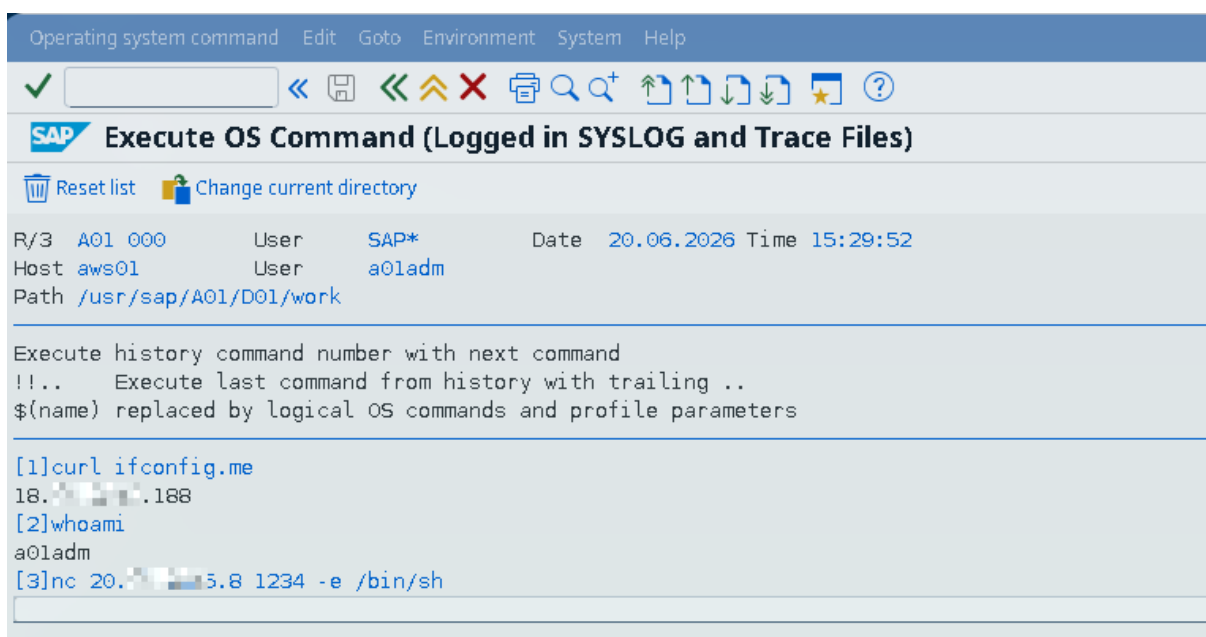


Figure 8: Command execution through RSBDCOS0, including validation of the execution context and reverse shell initiation.

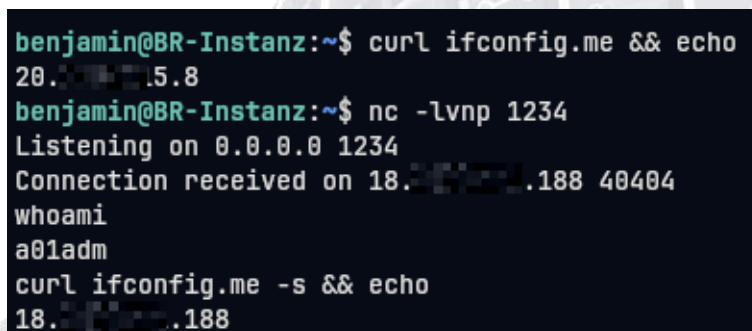
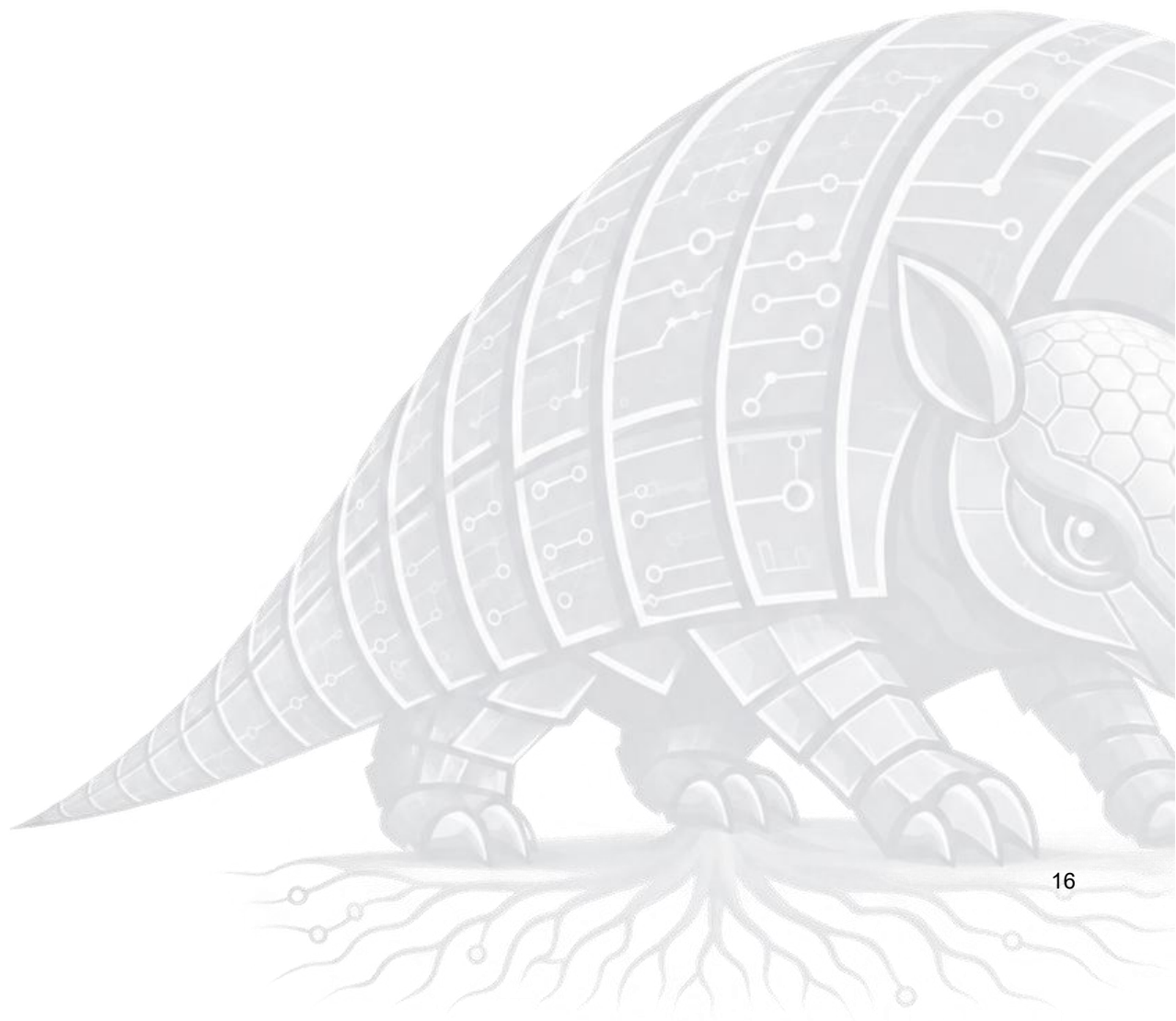


Figure 9: Reverse shell connection received on the attacker-controlled system, providing interactive access to the SAP host.

A reverse shell fundamentally changes the nature of an SAP compromise. While individual command execution already provides significant capabilities, a reverse shell grants continuous interactive access to the operating system. Attackers can execute arbitrary commands, upload and download files, perform reconnaissance, establish persistence mechanisms, and interact with the host in the same manner as a locally logged-in user.

From this point onward, the SAP application effectively becomes only the initial access vector. The attacker's focus shifts to the underlying operating system, connected infrastructure, and opportunities for privilege escalation. Depending on the permissions assigned to the SAP service account, the attacker may gain access to sensitive SAP data, operating system resources, connected databases, backup systems, or other enterprise services.

The example shown in this whitepaper uses a simple Netcat-based reverse shell for demonstration purposes. In a real-world attack, adversaries may employ more sophisticated techniques. The fundamental security concern remains the same: unrestricted operating system command execution from SAP can provide attackers with direct control over the host.



4. Defense in Depth and Blast Radius Reduction

4.1. Why Prevention alone is Insufficient

The primary objective of most security programs is to prevent compromise. Security hardening, vulnerability management, network segmentation, secure development practices, endpoint protection, and continuous monitoring all contribute to reducing the likelihood of successful attacks. These controls remain essential and should form the foundation of every SAP security strategy.

However, modern enterprise environments are complex, highly interconnected, and continuously evolving. New vulnerabilities are discovered regularly, misconfigurations occur, credentials are exposed, and attackers continuously develop new techniques to bypass existing protections. As a result, organizations must acknowledge the possibility that preventive controls may eventually fail.

This reality has led to the widespread adoption of defense-in-depth strategies. Rather than relying on a single security boundary, multiple independent layers of protection are deployed throughout the environment. If one control fails, additional controls remain available to limit the attacker's progress.

Within SAP environments, this principle becomes particularly important due to the extensive trust relationship between SAP applications and the underlying operating system. Once an attacker gains the ability to execute operating system commands or perform file-system operations through the SAP stack, traditional preventive controls have already been bypassed. At this stage, the relevant question is no longer how to prevent compromise, but how to limit its consequences.

Reducing the impact of a successful compromise therefore becomes an important security objective alongside prevention and detection.

4.2. The Philosophy of the Following Guidance

The following guidance is built around a simple assumption: compromise is possible, but its impact can and should be constrained.

Traditional operating-system security models often rely heavily on discretionary access controls. Once an attacker obtains the privileges of a trusted account, the operating system typically permits any action that account is authorized to perform. In SAP environments, this can provide broad access to files, processes, and resources that were never intended to be exposed to an attacker.

The goal is to introduce additional security boundaries that continue to enforce restrictions even after a trusted account has been compromised. These controls are designed to reduce an attacker's freedom of action, contain malicious activity, and limit opportunities for escalation and lateral movement.

Rather than attempting to eliminate all security risk, the objective is to reduce blast radius. A successful compromise of an SAP system should remain a localized security incident rather than becoming the starting point for a wider infrastructure compromise.

4.3. Security Objectives

The security controls discussed in this paper are intended to support four primary objectives.

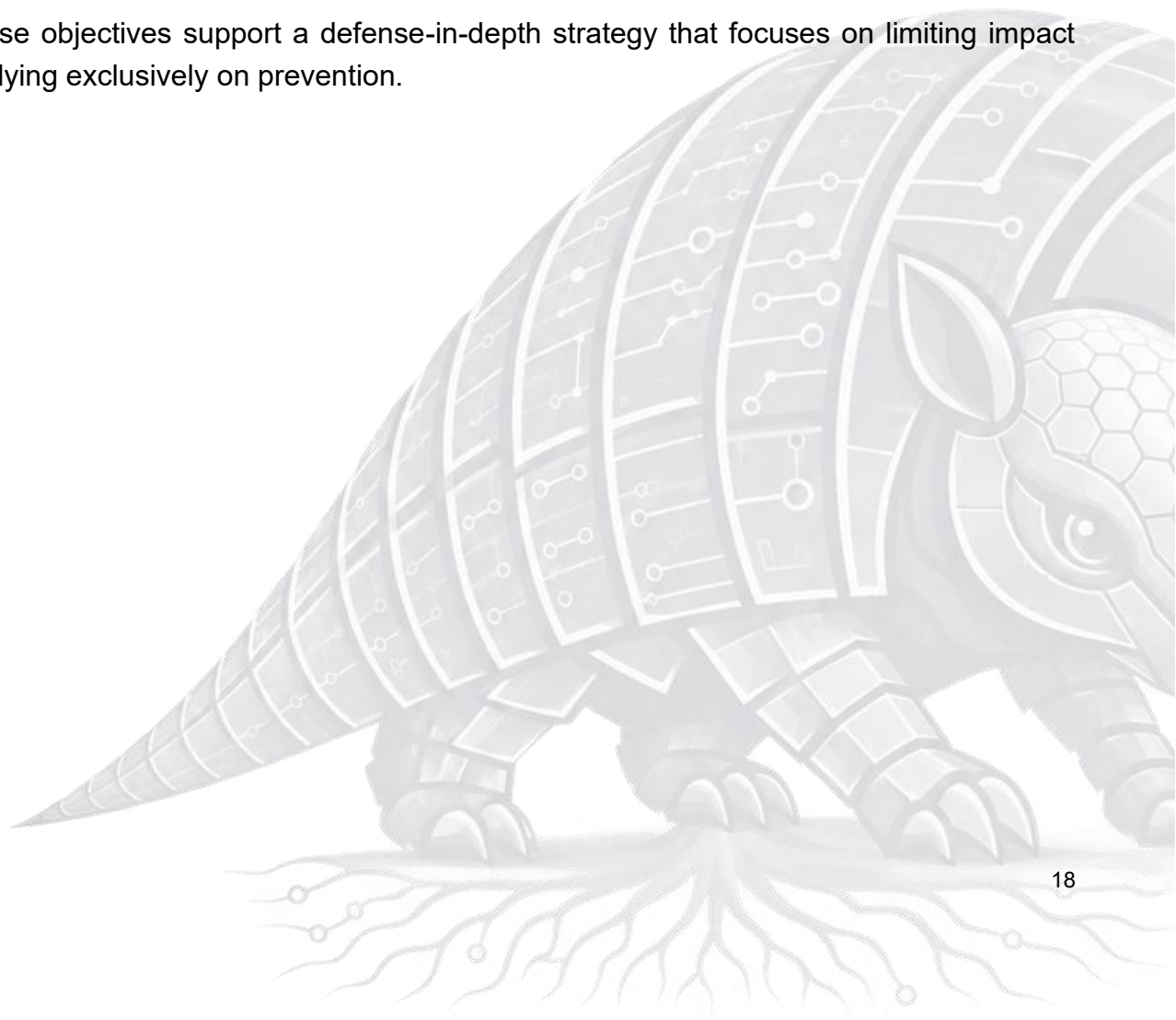
The first objective is to limit file-system access. A compromised SAP account should only be able to access files and directories that are required for legitimate operation. Access to unrelated operating-system resources, sensitive credentials, and administrative data should be restricted wherever possible.

The second objective is to limit process execution. Attackers frequently rely on additional tools, scripts, and binaries after obtaining access to a system. Restricting which executables may run significantly reduces the attacker's ability to perform reconnaissance, establish persistence, or execute malicious payloads.

The third objective is to reduce opportunities for privilege escalation and lateral movement. Access to shared resources, administrative tooling, and trusted communication paths should be minimized to prevent attackers from expanding beyond the initially compromised system.

The fourth objective is to improve containment. Even if an attacker successfully compromises an SAP operating-system account, the resulting activity should remain constrained to a narrowly defined operational boundary.

Together, these objectives support a defense-in-depth strategy that focuses on limiting impact rather than relying exclusively on prevention.



5. Security Risk Mitigation Strategies

The security objectives described in the previous chapter can be implemented using a combination of operating-system-level controls. No individual mechanism provides complete protection. Instead, multiple layers of enforcement work together to restrict what a compromised SAP account can access, modify, or execute.

The controls discussed in this chapter are not intended to replace existing SAP security measures. Rather, they complement traditional hardening approaches by providing additional protection at the Linux operating-system layer.

5.1. File System Permissions and Ownership

Traditional Linux security is primarily based on discretionary access controls (DAC). Access decisions are determined by file ownership, group membership, and permission bits assigned to files and directories.

Correctly designed file-system permissions remain an important security foundation. Sensitive files should be accessible only to the users and groups that require them. Administrative accounts should be separated wherever possible, shared permissions should be minimized, and unnecessary access to system resources should be removed.

Within SAP environments, careful attention should be paid to shared file systems, transport directories, backup locations, integration scripts, and administrative tooling. Excessive permissions granted to SAP-related accounts can significantly increase the potential impact of a compromise.

While DAC remains an essential security mechanism, it has inherent limitations. Once an attacker operates within the security context of a trusted account, the operating system generally treats all actions performed by that account as legitimate. Additional controls are therefore required to provide stronger containment.

5.2. SELinux

Security-Enhanced Linux (SELinux) extends the traditional Linux security model by implementing Mandatory Access Control (MAC). Unlike discretionary permissions, which are controlled by users and file owners, SELinux enforces centrally defined security policies at the kernel level.

Every process and file on the system is assigned a security context. When a process attempts to access a resource, the Linux kernel evaluates the requested action against the active SELinux policy before access is granted.

This approach enables significantly more granular control over system behavior. Policies can restrict which files a process may access, which network connections it may establish, which resources it may modify, and which interactions are permitted between different system components.

From an SAP security perspective, SELinux provides process containment. Even if an attacker gains control over an SAP-related process or operating-system account, access to resources outside the defined security policy can still be denied. The resulting restrictions help prevent unauthorized access to sensitive operating-system resources and reduce opportunities for post-compromise expansion. Access attempts that violate the policy are denied by the kernel itself, significantly limiting the attacker's ability to expand beyond the intended operational boundary.

This makes SELinux a particularly effective containment mechanism for reducing the impact of operating-system-level compromise and limiting the blast radius of successful attacks.

If permission is denied, an "avc: denied" message will be recorded in /var/log/audit/audit.log or /var/log/messages, as in the following example:

```
1 type=AVC msg=audit(1768559236.773:3021): avc: denied { write } for pid=2896
comm="sapstartsrv" name="0.3_50013_50014_12288_2_06" dev="dm-7" ino=2622517
scontext=system_u:system_r:init_t:s0 tcontext=system_u:object_r:usr_t:s0 tclass=file
permissive=1
```

Listing 2: Example SELinux AVC denial event

5.3. fapolicyd

While SELinux focuses on controlling behaviour, fapolicyd focuses on controlling execution.

fapolicyd is a file access policy framework that allows administrators to define which executables are permitted to run on a system. Execution decisions can be based on file trust, integrity information, package management metadata, and administrator-defined policies.

A common technique used during post-compromise activity is the introduction of additional tooling. Attackers frequently upload scripts, utilities, tunneling tools, malware, or custom binaries to support their operations. Traditional Linux systems generally allow execution of such files provided the account possesses sufficient permissions.

fapolicyd introduces an additional layer of control by allowing execution only from trusted sources. Binaries installed through approved package repositories, trusted SAP software, and explicitly approved applications can be permitted while unknown or untrusted executables are blocked.

This significantly reduces an attacker's ability to introduce external tooling and supports a stronger application allowlisting model.

5.4. How the Controls Work Together

Each control discussed in this chapter addresses a different aspect of post-compromise activity.

File-system permissions establish the foundational access model and define which users and groups may access specific resources. SELinux adds a mandatory enforcement layer that restricts process behavior even when traditional permissions would allow access. fapolicyd complements both mechanisms by controlling which software is permitted to execute on the system.

Together, these controls provide multiple independent security boundaries. An attacker who successfully compromises an SAP operating-system account must overcome several layers of enforcement before accessing sensitive resources, executing unauthorized code, or expanding beyond the compromised system.

Security Objective	DAC	SELinux	fapolicyd
Restrict file access	◐	●	○
Restrict process behaviour	○	●	○
Restrict executable launch	○	◐	●
Prevent uploaded malware execution	○	◐	●
Contain compromised SAP processes	◐	●	◐

Legend: ● = Strong / Primary Control ◐ = Partial / Indirect Control ○ = Not Applicable / No Significant Coverage

Table 1: Comparison of Linux Security Controls for SAP Blast-Radius Reduction

5.5 From Guidance to Pilot Implementation

The controls presented in this whitepaper provide a framework for strengthening operating-system security boundaries in SAP landscapes. They are intended to help organizations identify relevant security objectives, assess suitable Linux controls, and define an implementation approach appropriate to their environment.

The SELinux policies developed as part of Project Ironroot have so far been created and validated using defined test scenarios in test environments. As a next step, we would like to gain experience with them in environments that more closely reflect real-world production operations before making them more widely available.

We are therefore looking for organizations interested in joining us in a pilot project. Pilot participants receive access to the policies to evaluate and test them in their own landscapes, while helping us to further refine and validate the approach.

The policies reflect the architecture, components, and operational requirements of the original test environment. As SAP landscapes differ in their installed components, release levels, integrations, custom developments, file-system structures, and administrative workflows, the policies are best seen as a technical starting point rather than a ready-to-deploy configuration.

The pilot gives participants the opportunity to determine how well the policies suit their own SAP landscapes. Feedback and findings from participating environments will contribute directly to the further development of the Ironroot approach.

Organizations interested in participating can contact the Project Ironroot team through our [contact form](#). The following chapter describes the technical principles and implementation findings that form the basis for evaluating the policies.

6. Technical Implementation of SELinux and fapolicyd

The implementation focused primarily on SELinux and fapolicyd, complemented by traditional Linux discretionary access controls where appropriate. The tests were performed on an S/4HANA environment running on RHEL 9/10 and covered the attack scenarios previously demonstrated through SAP: arbitrary file read, arbitrary file write, execution of operating-system binaries, establishment of a reverse shell.

The results show that no single operating-system control addresses all of these attack paths. Instead, the most effective implementation combines:

- an SAP-specific SELinux policy to separate SAP components and restrict access to operating-system and SAP resources
- SELinux user mappings to extend those restrictions from service processes to interactive SAP administrator sessions
- fapolicyd to prevent execution of untrusted binaries and scripts
- minimization of installed operating-system tools and packages
- conventional file permissions and ACLs where they can be applied without affecting SAP functionality
- firewalld together with SELinux network controls to provide independent layers of network restriction

This results in a layered enforcement model in which compromise of an SAP application process or SAP administrative account does not automatically provide unrestricted access to the remaining SAP landscape or operating system.

6.1. SAP-Specific SELinux Policy

Running SELinux in enforcing mode alone is not sufficient to meaningfully isolate SAP if the SAP processes continue to execute in an unconfined SELinux domain. During testing, SAP processes running under an unconfined policy remained able to access files whenever normal Linux ownership and permission bits permitted it. This applied, for example, to files accessed through transaction AL11 or ABAP OPEN DATASET.

To introduce effective mandatory access controls, dedicated SELinux policy modules were created for the SAP landscape. The implementation consisted of five sample modules:

- `sap_general`, containing shared SAP file types and infrastructure paths;
- `sap_hostctrl`, covering the SAP Host Agent;
- `sap_hana`, covering SAP HANA database instances;
- `sap_netweaver`, covering ABAP, Java, ASCS, and ERS application instances; and
- `sap_hostctrl_passwd.cil`, containing the `passwd rootok` rule required by `saphostexec` to use `su` when switching to `<sid>adm` users.

The policy assigns dedicated SELinux types to SAP resources and executes different SAP components within dedicated domains. Permissions can therefore be granted based on the actual functional requirements of the respective SAP component rather than solely on Unix users, groups, and mode bits.

This distinction is particularly important in standard SAP installations because multiple SAP administrators commonly belong to the `sapsys` group. Traditional discretionary access control therefore frequently gives one SAP component access to resources belonging to another component. SELinux provides an additional mandatory control layer capable of denying such access even where Unix permissions permit it.

6.2. Extending Enforcement to SAP Administrator Sessions

Initially, SELinux restrictions protected only SAP processes started through `systemd`. This still left an important gap: an attacker who obtained an interactive session as an SAP administrator could perform actions directly from the shell outside the confined SAP process domain.

To close this gap, dedicated SELinux users were created for the HANA and NetWeaver administrator accounts and mapped to the corresponding Linux users. As a result, the same restrictions applied both to operations performed through SAP and to commands executed from an interactive shell.

This changes the security boundary substantially. A compromised `<sid>adm` account is no longer equivalent to an unrestricted Unix account with all permissions granted through the `sapsys` group. The SELinux domain remains effective even after the attacker obtains shell access.

6.3. Mitigating Arbitrary File Read

Arbitrary file read cannot be eliminated universally through SELinux. Operating-system files such as `/etc/passwd` already carry SELinux labels required by the operating system itself. Because a filesystem object can carry only one SELinux type label, such objects cannot simply be relabeled for SAP without potentially disrupting other system functionality. During remediation testing, this prevented a clean policy-based restriction of every system file that might be readable by an SAP process. In particular, restricting access to `/etc/passwd` from ABAP was not found to be practicable.

This is an important limitation of the implementation. SELinux is highly effective where SAP-owned resources can be assigned dedicated types, but it cannot automatically provide complete isolation from all globally required operating-system resources.

Traditional File Access Control Lists (ACLs) can theoretically provide additional restrictions for individual operating-system files. Testing, however, demonstrated that this must be approached carefully. Revoking access to `/etc/passwd` from the SAP administrator prevented the SAP dialog instance from starting. Certain SAP kernel operations may therefore depend on access even where the administrator itself should ideally not require it.

Consequently, operating-system file access should be reduced selectively rather than through broad deny rules. A practical implementation approach is to identify the files actually accessed by SAP processes during normal operation - for example through targeted tracing - and then evaluate which remaining files can safely be restricted. One proposed method is:

```
1 strace -f -e trace=/open -o /var/tmp/${PID}-open.strace -p ${PID}D
```

Listing 3: strace command proposed for SAP file-access baseline analysis

`${PID}` in this instance needs to be replaced by SAPs PIDs. This can help establish a functional baseline before additional access controls are introduced.

A concrete example demonstrated the value of separating HANA and NetWeaver resources.

Without the custom SELinux policy, the NetWeaver administrator could read a HANA file:

```
1 h89adm@host01:/usr/sap/H89/home> cd /hana/log/H89/mnt00001/hdb00001
2 h89adm@host01:/hana/log/H89/mnt00001/hdb00001> ls -aldZ .
3 drwxr-x---. 2 h89adm sapsys unconfined_u:object_r:usr_t:s0 4096 Jul 16 04:23 .
4 h89adm@host01:/hana/log/H89/mnt00001/hdb00001> ls -lZ landscape.id
5 -rw-r-----. 1 h89adm sapsys unconfined_u:object_r:usr_t:s0 36 Jul 16 04:20 landscape.id
6 h89adm@host01:/hana/log/H89/mnt00001/hdb00001> awk '{print}' landscape.id
7 9a259d5e-0c01-405b-a1e3-5c93805983eb
8 a01adm@host01:/usr/sap$ cd /hana/log/H89/mnt00001/hdb00001
9 a01adm@host01:/hana/log/H89/mnt00001/hdb00001$ awk '{print}' landscape.id
10 9a259d5e-0c01-405b-a1e3-5c93805983eb
11
```

Listing 4: Pre-policy HANA file access from NetWeaver administrator accounts

The directory and file permissions permitted access through the shared sapsys group. Both the HANA administrator and the NetWeaver administrator were therefore able to read the file.

After applying the SAP-specific SELinux policy and labeling the HANA log resources with `sap_hana_log_t`, the NetWeaver administrator was denied access to the directory entirely:

```
1 a01adm@host01:/usr/sap$ cd /hana/log/H89/mnt00001/hdb00001
2 bash: cd: /hana/log/H89/mnt00001/hdb00001: Permission denied
```

Listing 5: Post-policy denial of NetWeaver access to HANA log resources

The same restriction applied to an ABAP program attempting to access the file through OPEN DATASET.

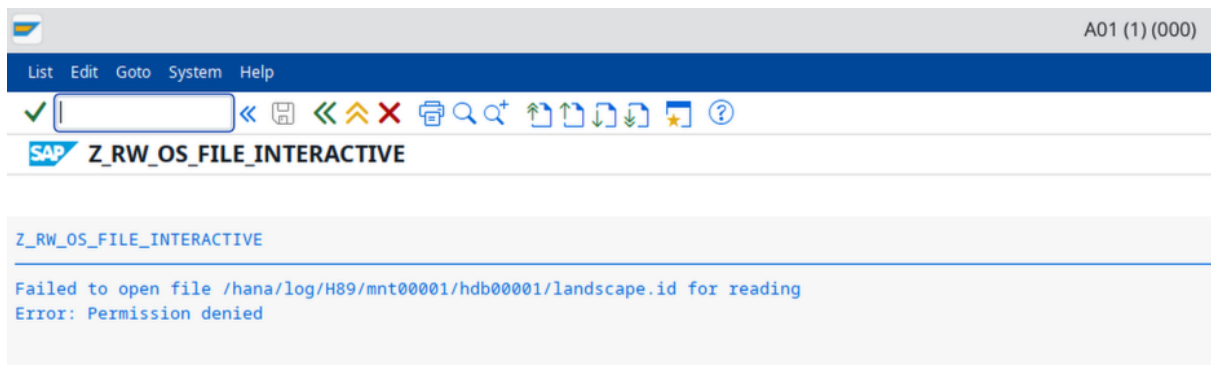


Figure 10: SELinux blocks an ABAP OPEN DATASET read of a HANA resource

This demonstrates an important Ironroot principle: resources belonging to one SAP component should not automatically become accessible to another component merely because both operating-system users belong to the same Unix group.

SELinux provides a mechanism for introducing this additional separation without redesigning SAP's native user and group structure.

6.4. Mitigating Arbitrary File Write

The same distinction between discretionary and mandatory controls applies to file creation and modification.

With SAP running in an unconfined SELinux domain, an ABAP program can write to directories such as `/tmp` wherever the SAP administrator could also write using normal operating-system permissions.

The custom SELinux implementation introduces different temporary-file types for different SAP domains. For example, a file created by the NetWeaver administrator in `/tmp` or `/var/tmp` receives the type:

```
1 sap_nw_u:object_r:sap_nw_tmp_t:s0
```

The HANA administrator can be permitted to read such a file while being denied permission to modify or execute it. Testing confirmed that the HANA administrator could read a file created by the NetWeaver account but received Permission denied when attempting to append data to it.

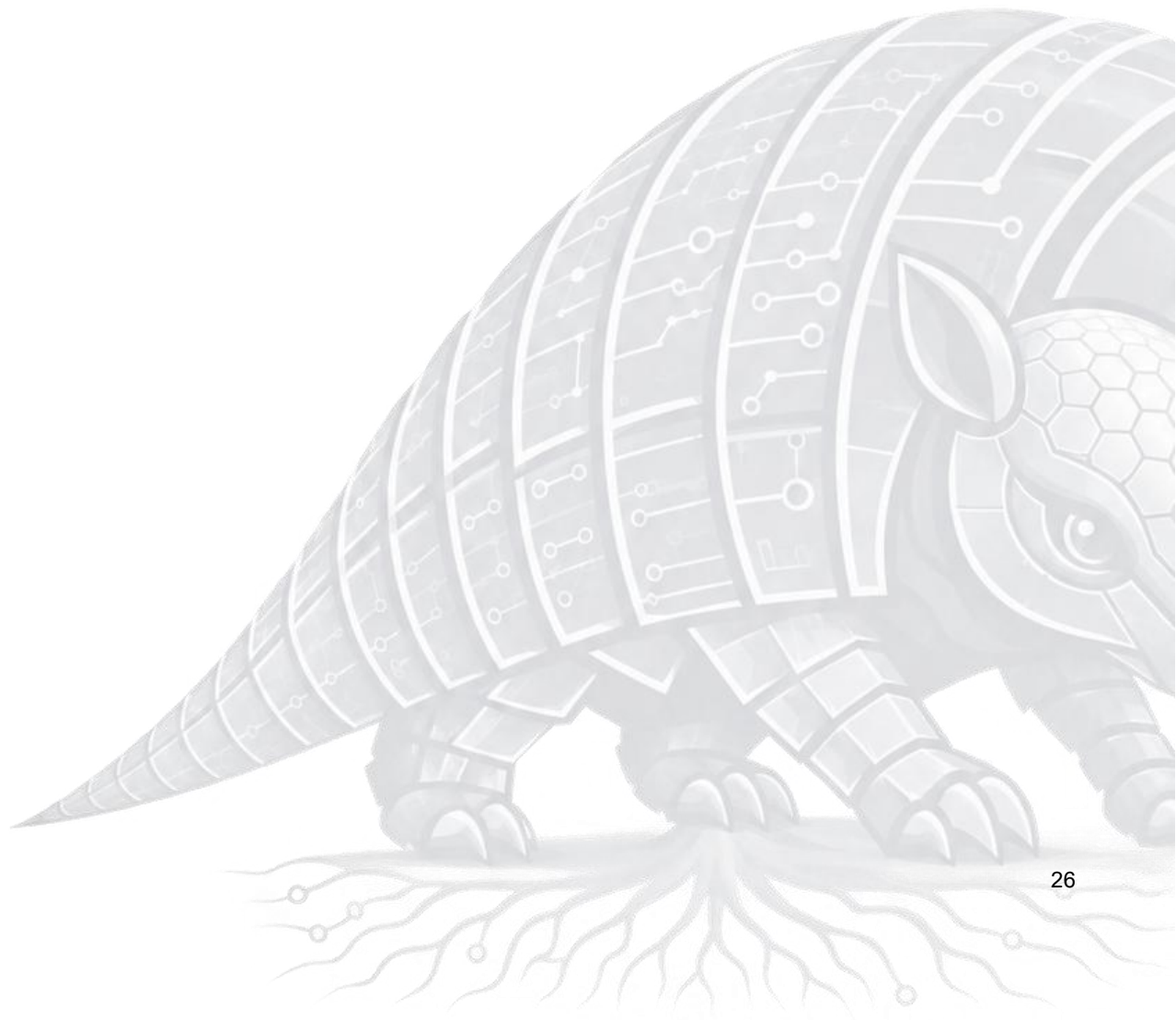
```

1 a01adm@host01:/home/a01adm$ cd /var/tmp
2 a01adm@host01:/var/tmp$ echo "Created by a01adm" > testfile-from-a01adm-shell.txt
3 <ctrl>d
4 a01adm@host01:/var/tmp$ ls -lZ testfile-from-a01adm-shell.txt
5 -rw-r--r--. 1 a01adm sapsys sap_nw_u:object_r:sap_nw_tmp_t:s0 18 Jul 17 12:56 testfile-
  from-a01adm-shell.txt
6 a01adm@host01:/var/tmp$ cat testfile-from-a01adm-shell.txt
7 Created by a01adm
8 h89adm@host01:/usr/sap/H89/HDB44> cd /var/tmp
9 h89adm@host01:/var/tmp> cat testfile-from-a01adm-shell.txt
10 Created by a01adm
11 h89adm@host01:/var/tmp> echo "test" >> testfile-from-a01adm-shell.txt
12 -sh: testfile-from-a01adm-shell.txt: Permission denied

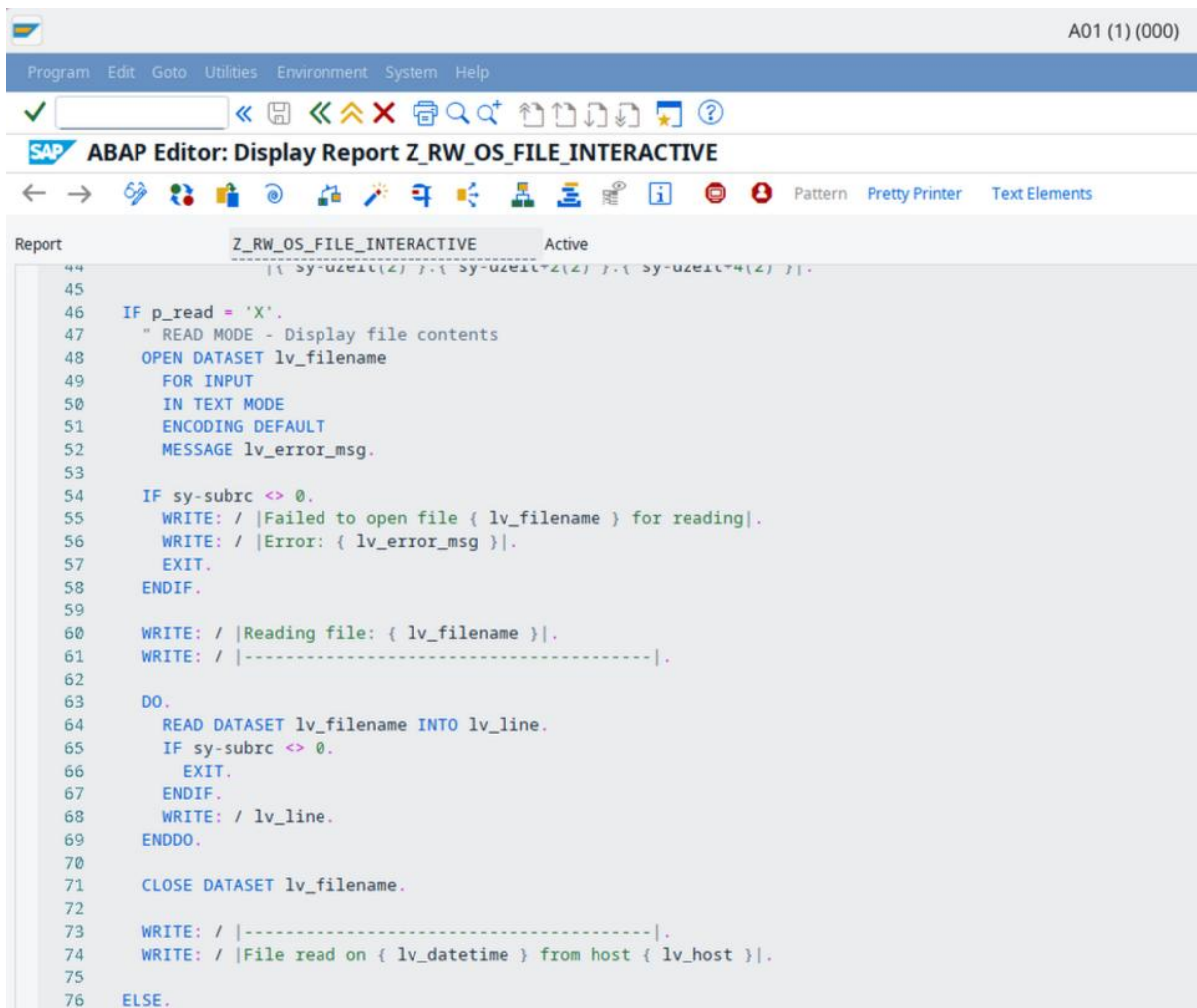
```

Listing 6: SELinux temporary-file labeling and cross-domain write-denial test

The reverse case was tested as well. Files created by the HANA administrator received the corresponding `sap_hana_tmp_t` type. The NetWeaver administrator remained able to read the file but was unable to overwrite it, both through an ABAP OPEN DATASET operation and from an interactive shell.

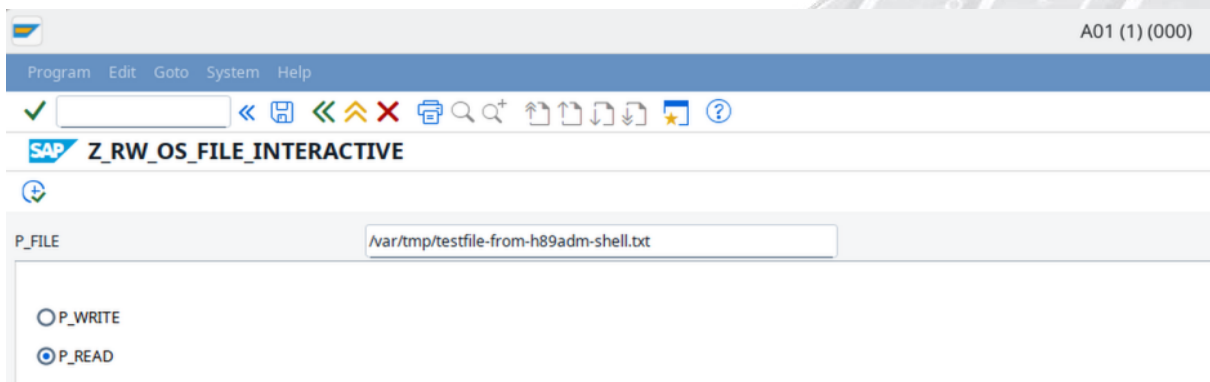


ABAP program: Reading (permitted)



```
44  
45  
46 IF p_read = 'X'.  
47   " READ MODE - Display file contents  
48   OPEN DATASET lv_filename  
49     FOR INPUT  
50     IN TEXT MODE  
51     ENCODING DEFAULT  
52     MESSAGE lv_error_msg.  
53  
54   IF sy-subrc <> 0.  
55     WRITE: / |Failed to open file { lv_filename } for reading|.  
56     WRITE: / |Error: { lv_error_msg }|.  
57     EXIT.  
58   ENDIF.  
59  
60   WRITE: / |Reading file: { lv_filename }|.  
61   WRITE: / |-----|.  
62  
63   DO.  
64     READ DATASET lv_filename INTO lv_line.  
65     IF sy-subrc <> 0.  
66       EXIT.  
67     ENDIF.  
68     WRITE: / lv_line.  
69   ENDDO.  
70  
71   CLOSE DATASET lv_filename.  
72  
73   WRITE: / |-----|.  
74   WRITE: / |File read on { lv_datetime } from host { lv_host }|.  
75  
76 ELSE.
```

Figure 11: ABAP OPEN DATASET logic used to test file read access



Program Edit Goto System Help

✓ << <<< >>> >> > ?

SAP Z_RW_OS_FILE_INTERACTIVE

P_FILE

☐ P_WRITE
☒ P_READ

Figure 12: ABAP test program configured for a read operation against a temporary file

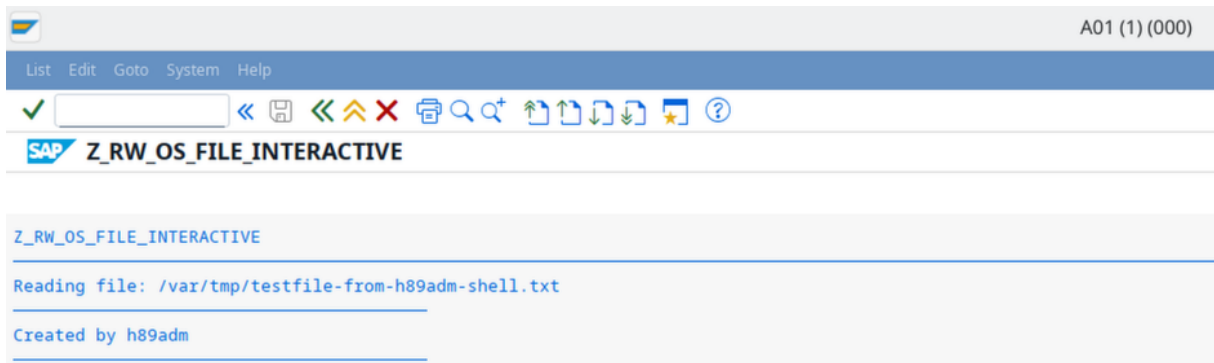


Figure 13: Successful ABAP read of a temporary file created by the HANA administrator

ABAP program: Writing (blocked)

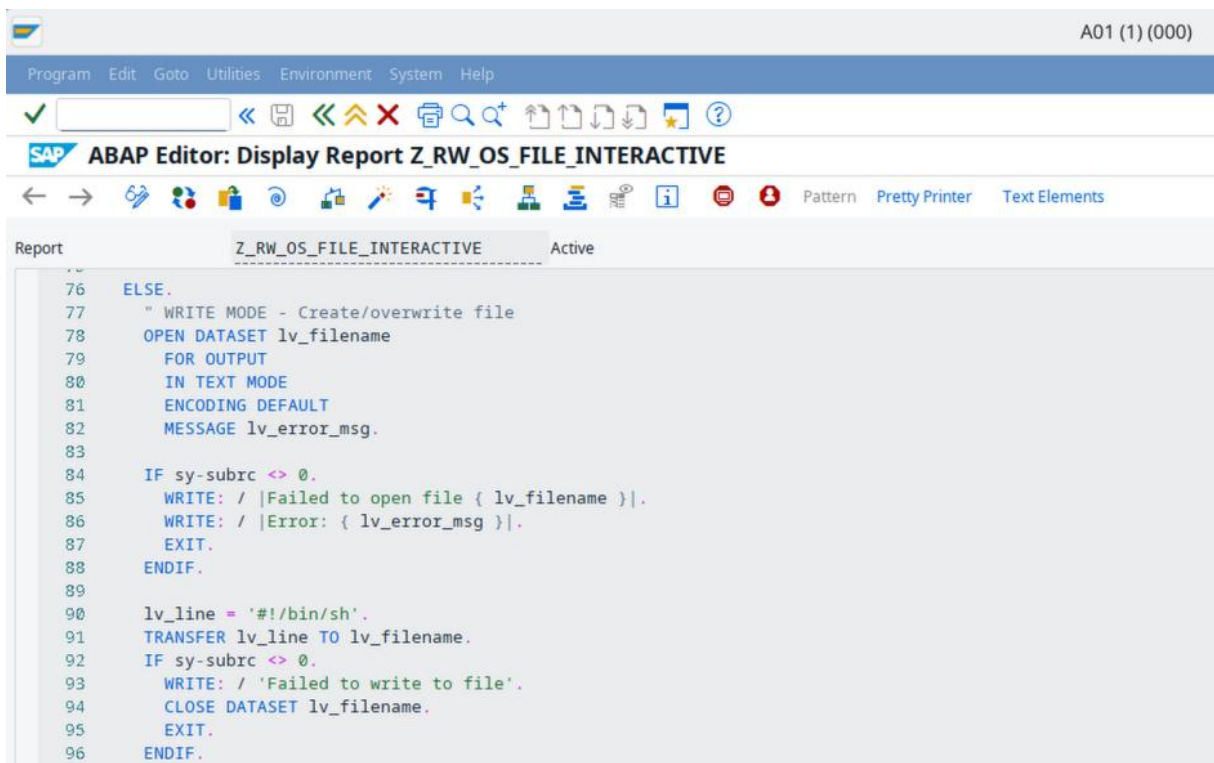


Figure 14: ABAP OPEN DATASET logic used to test file write access

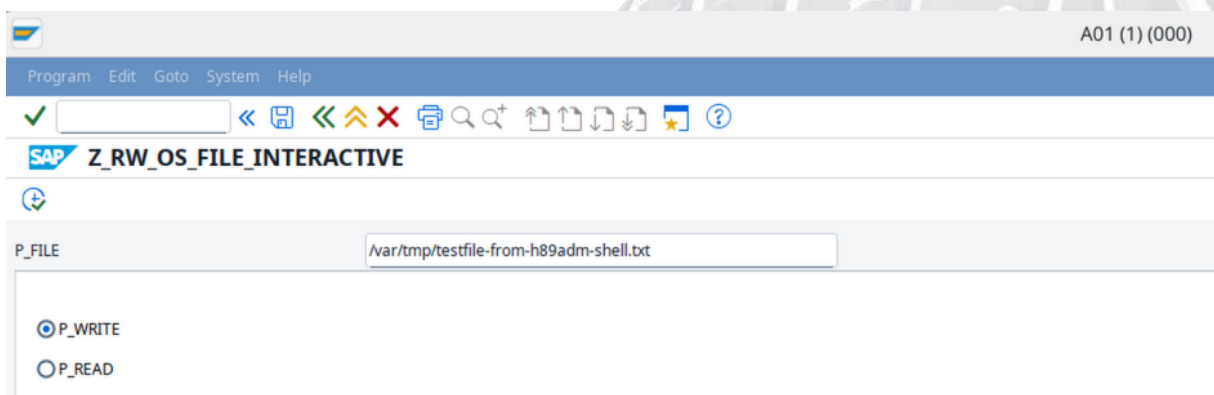


Figure 15: ABAP test program configured for a write operation against a temporary file

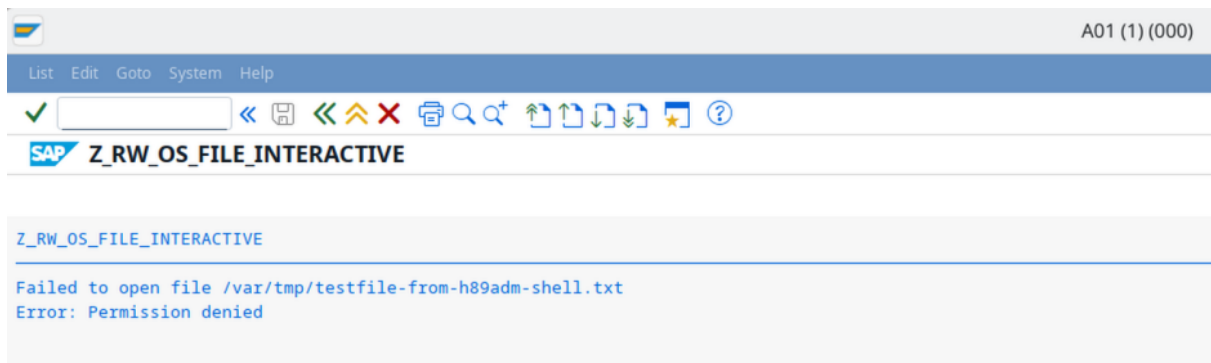


Figure 16: ABAP write operation blocked with Permission denied under the SELinux policy

This provides a useful containment mechanism even for globally writable directories. Rather than attempting to make /tmp itself unavailable to SAP, which proved incompatible with normal SAP operation, the security policy controls what different SAP domains may subsequently do with objects created there.

ACLs can additionally restrict write access to selected operating-system directories. However, testing again demonstrated the danger of applying these restrictions too broadly. Removing write access to /tmp for the SAP administrator caused the SAP dialog instance to fail during startup.

The recommended implementation is therefore to use SELinux as the primary separation mechanism and apply ACL restrictions only to specifically identified directories whose restriction has been validated against the SAP workload.

In our example, the SAP HANA database administrator creates a file in /tmp or /var/tmp (or any other directory labeled tmp_t), using a terminal session. Due to the definition in the SELinux policies, this file then automatically gets the label sap_hana_u:object_r:sap_hana_tmp_t:s0. The SAP NetWeaver administrator tries to read it (which is allowed by the policy) and also to overwrite it (which is forbidden), using OPEN DATASET in ABAP program and from the command line (terminal session).

fapolicyd does not mitigate arbitrary file creation or modification and should not be treated as a substitute for filesystem access controls.

6.5. Restricting Execution

Executable control represents one of the strongest results of the remediation exercise.

Files copied to a compromised SAP server should not automatically become executable simply because normal Unix execute permissions have been assigned. fapolicyd provides an application allowlisting mechanism that distinguishes between trusted and untrusted executable content.

Operating-system packages are considered trusted through the package-management trust database. Additional applications can be explicitly entered into fapolicyd trust files. The trust entry records the executable path together with its size and SHA-256 checksum. Programs not belonging to the operating system or explicitly trusted software can therefore be prevented from executing.

Applying fapolicyd rules and testing them from inside the SAP environment as shown in 3.4.4. we execute the following steps:

1. In a terminal, check if the fapolicyd service is running and enable it if necessary:

```
1 # systemctl status fapolicyd
```

2. Enable and start fapolicyd if required.

```
1 # systemctl enable fapolicyd --now
```

3. In transaction SE38, enter report RSBDCOS0 and execute it with F8.

4. Execute the previously created script and press Return.

```
1 sh /var/tmp/hello.sh
```

5. Confirm that execution is denied.

```
1 RC = 126
```

6. Add the script to the trust database.

```
1 # fapolicyd-cli --file add /var/tmp/hello.sh --trust-file trusted_programs
```

7. Restart fapolicyd.

```
1 # systemctl restart fapolicyd
```

8. Repeat the execution test and confirm that "Hello from the OS" is displayed.

Listing 7: fapolicyd enablement and trust-database validation commands

This directly addresses an important post-exploitation technique: uploading an attacker-controlled tool or script and executing it from a writable directory.

6.6. Reducing the Executable Attack Surface

Execution control should be combined with package minimization.

For new systems, only software required by the SAP landscape should be installed. The tested recommendations include either starting with the RHEL Server software environment recommended by SAP or evaluating whether a Minimal Environment installation can be used and adding only packages required by applicable SAP Notes or identified as dependencies during installation and testing.

Existing systems should similarly be reviewed for unnecessary packages. Tools required only temporarily for troubleshooting or administration can be installed when necessary and removed afterwards.

This reduces both the number of executable programs available to an attacker and the size of the fapolicyd trust base.

6.7. Limiting Reverse-Shell Capabilities

An operating-system reverse shell represents a significant transition in an attack because it gives the attacker an interactive environment from which additional reconnaissance, tooling, persistence, and lateral movement can be performed.

The remediation approach addresses this at two levels.

First, unnecessary programs capable of establishing such connections should be removed. The `nc` utility used in the original attack scenario is provided by the RHEL `nmap-ncat` package. Where it is not operationally required, it can be removed.

Second, `fapolicyd` prevents an attacker from simply uploading an alternative reverse-shell binary or script and executing it.

This does not imply that `fapolicyd` alone makes reverse shells impossible. Trusted operating-system executables may themselves provide functionality that can potentially be misused, such as the aforementioned `nmap-ncat`, or others like `Bash` or `Python`. The control nevertheless materially reduces the attacker's available toolset and prevents straightforward execution of arbitrary post-exploitation tooling.

The result is therefore best understood as attack-surface reduction and containment, rather than absolute prevention.

`firewalld` remains the conventional mechanism for controlling network access, but firewall configurations can be disabled or modified accidentally. SELinux provides an independent enforcement layer controlling whether a process domain may bind to particular port types. SELinux denials additionally create operating-system audit events, providing both preventative and detective value.

It is important to note that the defense mechanisms provided by `fapolicyd` and SELinux alone are not enough to prevent reverse shell capabilities. Therefore conventional methods like hardened firewalls are necessary to implement. To ensure firewalls are best fit to prevent reverse shells, it is important to note that not only ingress connections are monitored and restricted, but also egress connections. While it is cumbersome to do so, reverse shells work specifically because of the lack of egress monitoring and restrictions - leaving that part out, will not prevent reverse shells.

6.8. Operational Considerations

The examples in the preceding sections demonstrate the security boundaries that can be created with SELinux, `fapolicyd`, conventional permissions, ACLs, and network controls. They should not be interpreted as a complete or directly deployable policy set. Their primary purpose is to show how security objectives can be translated into enforceable operating-system controls and how the effectiveness of those controls can be validated.

Production adoption should be treated as an iterative engineering process rather than a one-time policy deployment. The objective is to derive the smallest set of permissions required for normal SAP operation while retaining meaningful restrictions against post-compromise activity.

The remediation tests also demonstrate why an SAP SELinux policy cannot simply be generated by denying everything outside the SAP installation directories.

SAP processes interact with operating-system resources during normal operation. Files such as `/etc/passwd`, temporary directories such as `/tmp`, kernel interfaces, authentication infrastructure, and system utilities may be required by particular SAP components or administrative workflows.

Overly restrictive policy therefore creates a real availability security risk. This was observed directly when conventional filesystem permissions were modified: denying access to `/etc/passwd` or preventing the SAP administrator from writing to `/tmp` caused the SAP dialog instance to fail.

A production implementation should consequently follow an iterative process:

- establish the normal filesystem, process, and network behaviour of the SAP landscape
- deploy the SELinux policy initially in a controlled testing environment
- review AVC events for legitimate SAP operations
- grant only those permissions that are operationally required
- label SAP resources according to their component and function
- map SAP administrator accounts to confined SELinux users
- enable enforcing mode
- introduce `fapolicyd` after the required executable baseline has been established
- continuously review policy exceptions when SAP, the operating system, or third-party software is changed

The objective should not be to suppress SELinux denials indiscriminately. Every required exception effectively enlarges the permissions available to a compromised SAP process and should therefore be evaluated in the context of the intended security boundary.

6.9. Resulting Security Architecture

The remediation results support a layered implementation in which each native Linux control addresses a different part of the post-compromise attack chain.

SELinux provides segmentation and mandatory access control. It separates HANA, NetWeaver, SAP Host Agent, administrator sessions, file types, and network resources even where conventional Unix permissions would permit access.

`fapolicyd` provides executable allowlisting. It substantially limits the attacker's ability to introduce and execute new tooling after gaining write access to the system.

Package minimization reduces the set of already trusted tools. This is important because execution control is substantially less valuable if the operating system already contains a large collection of utilities suitable for post-exploitation.

Traditional permissions and ACLs provide additional restrictions where SAP compatibility permits them. Their applicability must, however, be validated carefully.

firewalld and SELinux provide complementary network controls. A firewall restricts network traffic, while SELinux controls whether the compromised process is permitted to create the corresponding socket in the first place.

Together, these controls materially alter the consequence of an SAP compromise. Successful execution of malicious ABAP code or compromise of an SAP administrator account no longer necessarily provides unrestricted access to other SAP components, arbitrary executables, network services, or root-level administration.

The implementation therefore follows the core Ironroot concept: assume that the application layer can be breached, and use the operating system as an independent security boundary to reduce the resulting blast radius.

7. Conclusion

This whitepaper demonstrated four practical attack paths through which SAP-level access can lead to direct interaction with the underlying operating system: reading and writing files, executing operating-system commands, and obtaining an interactive shell.

Although these capabilities are individually well known, their combined use creates a broader attack chain. File access can expose credentials and configuration data, file write access can enable payload staging or system manipulation, and command execution can establish an operating-system foothold. This highlights the importance of extending SAP security beyond the application layer.

The Ironroot approach adds independent enforcement boundaries at the operating-system level. Our tests showed that SELinux can separate SAP components and resources, restrict cross-domain file access, confine SAP processes and administrator sessions, and control access to privileged operating-system and network functionality. fapolicyd complements these controls by preventing untrusted binaries and scripts from being executed, even when an attacker has gained the ability to write files.

The strongest results are achieved through a layered architecture. SELinux and fapolicyd work alongside established SAP security measures, least-privilege configuration, package minimization, filesystem permissions and ACLs, host firewalling, network segmentation, and effective monitoring. Each layer addresses a different part of the attack chain and reduces the opportunities available after an initial compromise.

Because SAP landscapes differ in their components, integrations, and operational requirements, these controls should be introduced through an iterative, environment-specific process. Establishing a baseline of legitimate behavior, validating policies in a representative test

environment, and introducing enforcement incrementally enables organizations to strengthen containment while maintaining reliable SAP operations.

Project Ironroot provides a practical foundation for this process. The concepts, implementation patterns, and accompanying examples enable organizations to define security boundaries suited to their own SAP landscapes and to translate defense-in-depth principles into enforceable controls.

By treating the operating system as an active and independent security boundary, organizations can materially reduce the blast radius of an SAP compromise, protect neighboring components and host resources, and make post-exploitation significantly more difficult. The result is a more resilient SAP landscape in which security extends from the application layer to the underlying platform.

Contributors

PwC

Benjamin Fani Sani
Daniel Peisker

Fortinet

Julian Petersohn

Red Hat

Bernd Finger
Christoph Brune
Matthias Czwikla

No Monkey

Waseem Ajrab

